

网络爬虫概述

- 定义

网络蜘蛛(spider)、网络机器人, 抓取网络数据的程序

其实就是用Python程序模仿人点击浏览器并访问网站, 而且模仿的越像越好, 让Web站点无法发现你不是人

- 爬取数据目的

- 1、公司项目测试数据
- 2、公司业务部门及其他部门所需数据
- 3、数据分析

- 企业获取数据方式

- 1、公司自有数据
- 2、第三方数据平台购买(数据堂、贵阳大数据交易所)
- 3、爬虫爬取数据

- Python做爬虫优势

- 1、Python : 请求模块、解析模块丰富成熟,强大的Scrapy网络爬虫框架
- 2、PHP : 对多线程、异步支持不太好
- 3、JAVA: 代码笨重,代码量大
- 4、C/C++: 虽然效率高,但是代码成型慢

- 爬虫分类

- 1、通用网络爬虫(搜索引擎使用,遵守robots协议)

robots协议: 网站通过robots协议告诉搜索引擎哪些页面可以抓取,哪些页面不能抓取,通用网络爬虫需要遵守robots协议 (君子协议)

<https://www.taobao.com/robots.txt>

- 2、聚焦网络爬虫: 自己写的爬虫程序

- 爬虫爬取数据步骤

- 1、确定需要爬取的URL地址
- 2、由请求模块向URL地址发出请求,并得到网站的响应
- 3、从响应内容中提取所需数据

- 1、所需数据,保存
- 2、页面中有其他需要继续跟进的URL地址,继续第2步去发请求, 如此循环

爬取网站思路

响应内容中存在

- 1、确认抓取数据在响应内容中是否存在
- 2、分析页面结构, 观察URL地址规律
 - 1、大体查看响应内容结构, 查看是否有更改 -- (百度视频案例)
 - 2、查看页面跳转时URL地址变化, 查看是否新跳转 -- (民政部案例)
- 3、开始代码进行数据抓取

响应内容中不存在

- 1、确认抓取数据在响应内容中是否存在
- 2、F12抓包,开始刷新页面或执行某些行为,主要查看XHR异步加载数据包
 - 1、GET请求: Request Headers、Query String Paramters
 - 2、POST请求: Request Headers、FormData
- 3、观察查询参数或者Form表单数据规律,如果需要进一步抓包分析处理
 - 1、比如有道翻译的 salt+sign,抓取并分析JS做进一步处理
 - 2、此处注意请求头中的cookie和referer以及User-Agent
- 4、使用res.json()获取数据,利用列表或者字典的方法获取所需数据

多级页面数据抓取

整体思路

- 1、爬取一级页面,提取 所需数据+链接,继续跟进
- 2、爬取二级页面,提取 所需数据+链接,继续跟进
- 3、... ..

代码实现思路

- 1、所有数据最终都会在一级页面遍历每条数据时全部拿到
- 2、避免重复代码 - 请求、解析需定义函数

Chrome浏览器安装插件

安装方法

- 1、把下载的相关插件(对应操作系统浏览器)后缀改为 .zip
 - 2、解压,打开Chrome浏览器 -> 右上角设置 -> 更多工具 -> 扩展程序 -> 点开开发者模式
- #3、把相关插件文件夹拖拽到浏览器中,释放鼠标即可安装
- #3、有的插件直接拖拽 .zip 文件释放即可

需要安装插件

- 1、Xpath Helper: 轻松获取HTML元素的XPath路径
 - # 开启/关闭: Ctrl + Shift + x
 - # Firefox浏览器插件: checker
- 2、Proxy SwitchyOmega: Chrome浏览器中的代理管理扩展程序
- 3、JsonView: 格式化输出json格式数据

请求模块

(一) urllib模块

1.模块名及导入

- 1、模块名: urllib.request
- 2、导入方式:
 - 1、import urllib.request
 - 2、from urllib import request

2.常用方法详解

(1) urllib.request.urlopen()

- 作用

向网站发起请求并获取响应对象

- 参数

- 1、url: 需要爬取的URL地址
- 2、timeout: 设置等待超时时间,指定时间内未得到响应抛出超时异常

- 第一个爬虫程序 - 01_urlopen.py

打开浏览器,输入百度地址(<http://www.baidu.com/>),得到百度的响应

```
import urllib.request

# urlopen() : 向URL发请求,返回响应对象
response=urllib.request.urlopen('http://www.baidu.com/')
# 提取响应内容
html = response.read().decode('utf-8','ignore') #'ignore'参数,遇到无法解码的字符,跳过
# 打印响应内容
print(html)
```

- 响应对象 (response) 方法

- 1、bytes = response.read() # read()得到结果为 bytes 数据类型
 - 2、string = response.read().decode() # decode() 转为 string 数据类型
 - 3、url = response.geturl() # 返回实际数据的URL地址
 - 4、code = response.getcode() # 返回HTTP响应码
- # 补充
- 5、string.encode() # string -> bytes
 - 6、bytes.decode() # bytes(字节串) -> string

思考: 网站如何来判定是人类正常访问还是爬虫程序访问? ? ?

向测试网站: <http://httpbin.org/get> 发请求,查看自己请求头 - 响应内容
代码如下

```
import urllib.request as request
response = request.urlopen('http://httpbin.org/get')
html =response.read().decode('utf-8')
print(html)
```

html中的请求头headers如下

```
"headers": {
    "Accept-Encoding": "identity",
    "Host": "httpbin.org",
    "User-Agent": "Python-urllib/3.11"
```

```
},
```

发现请求头中User-Agent竟然是:Python-urllib/3.11!!!!!!!!!!!!!!!!!!!!!!

我们需要重构User-Agent,发请求时带着User-Agent过去,但是 `urlopen()` 方法不支持重构User-Agent,那我们怎么办? 请看下面的方法!!!

(2) `urllib.request.Request()`

- 作用

创建请求对象(包装请求, 重构User-Agent, 使程序更像正常人类请求)

- 参数

- 1、`url`: 请求的URL地址
- 2、`headers`: 添加请求头 (爬虫和反爬虫斗争的第一步)

- 使用流程

- 1、构造请求对象(重构User-Agent)

```
req = urllib.request.Request(  
    url='http://httpbin.org/get',  
    headers={'User-Agent': 'Mozilla/5.0'}  
)
```
- 2、发请求获取响应对象(`urlopen`)

```
res = urllib.request.urlopen(req)
```
- 3、获取响应对象内容

```
html = res.read().decode()
```

- 示例 - 02_Request.py

向测试网站 (<http://httpbin.org/get>) 发起请求, 构造请求头并从响应中确认请求头信息

```
import urllib.request as request  
  
#常用变量  
url = 'http://httpbin.org/get'  
headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 6.1; rv:2.0.1) Gecko/20100101  
Firefox/4.0.1'}  
#1. 创建请求对象  
req = request.Request(url=url, headers=headers)  
  
#2. 获取响应对象  
response = request.urlopen(req)  
  
#3. 读取内容  
html = response.read().decode()  
  
print(html)
```

(二) requests模块

安装

- Linux

```
sudo pip3 install requests
```

- Windows

方法一

进入cmd命令行 : `python -m pip install requests`

方法二

右键管理员进入cmd命令行 : `pip install requests`

requests.get()

- 作用

向网站发起请求,并获取响应对象

```
res = requests.get(url,headers=headers)
```

- 参数

- 1、url : 需要抓取的URL地址
- 2、headers : 请求头
- 3、timeout : 超时时间,超过时间会抛出异常

- 响应对象(res)属性

- 1、encoding : 响应字符编码
`res.encoding = 'utf-8'`
- 2、text : 响应内容-字符串
- 3、content : 响应内容-字节流
- 4、status_code : HTTP响应码
- 5、url : 实际数据的URL地址

- 查询参数-params

字典,字典中键值对作为查询参数

- 使用方法

```
1、res = requests.get(url,params=params,headers=headers)
```

2、特点:

- * url为基准的url地址,不包含查询参数
- * 该方法会自动对params字典编码,然后和url拼接

- 示例

```
import requests

baseurl = 'http://tieba.baidu.com/f?'
params = {
    'kw' : '赵丽颖吧',
    'pn' : '50'
}
headers = {'User-Agent' : 'Mozilla/4.0'}
# 自动对params进行编码,然后自动和url进行拼接,去发请求
res = requests.get(url=baseurl,params=params,headers=headers)
res.encoding = 'utf-8'
print(res.text)
```

- Web客户端验证参数-auth

- 作用及类型

- 1、针对于需要web客户端用户名密码认证的网站
- 2、auth = ('username', 'password')

- 达内code课程方向案例

```
# xpath表达式
//a/@href
# url
http://code.tarena.com.cn/AIDCode/aid1904/14-redis/
```

思考：爬取具体的笔记文件？

```
import os

# 保存在: /home/tarena/redis
# 先判断 /home/tarena/redis 是否存在
1、不存在: 先创建目录,然后再保存 .zip
2、存在: 直接保存 .zip

# 使用频率很高
if not os.path.exists('路径'):
    os.makedirs('路径')
```

- SSL证书认证参数-verify

- 适用网站及场景

- 1、适用网站：https类型网站但是没有经过 证书认证机构 认证的网站
- 2、适用场景：抛出 `SSLError` 异常则考虑使用此参数

- 参数类型

```
1、verify=True(默认)    : 检查证书认证
2、verify=False(常用)   : 忽略证书认证
# 示例
response = requests.get(
    url=url,
    params=params,
    headers=headers,
    verify=False
)
```

- 代理参数-proxies

- 定义

- 1、定义：代替你原来的IP地址去对接网络的IP地址。
- 2、作用：隐藏自身真实IP,避免被封。

普通代理

- 获取代理IP网站

西刺代理、快代理(<https://www.kuaidaili.com/free/dps/4/>)、全网代理、代理精灵、
https://www.89ip.cn/index_6.html
<http://www.ip3366.net/?stype=1&page=2>
<http://www.66ip.cn/index.html>
<https://www.xsdaili.cn/dayProxy/ip/2459.html>
<https://ip.ihuan.me/>
<http://www.kxdaili.com/dailiip/1/1.html>

- 参数类型

- 1、语法结构

```
proxies = {
    '协议': '协议://IP:端口号'
}
```

- 2、示例

```
proxies = {
    'http': 'http://IP:端口号',
    'https': 'https://IP:端口号'
}
```

- 示例

使用免费普通代理IP访问测试网站: <http://httpbin.org/get>

```
import requests

url = 'http://httpbin.org/get'
headers = {
    'User-Agent': 'Mozilla/5.0'
}
# 定义代理,在代理IP网站中查找免费代理IP
proxies = {
    'http': 'http://112.85.164.220:9999',
    'https': 'https://112.85.164.220:9999'
}
html = requests.get(url,proxies=proxies,headers=headers,timeout=5).text
print(html)
```

思考: 建立一个自己的代理IP池, 随时更新用来抓取网站数据

- 1、从西刺代理IP网站上, 抓取免费代理IP
- 2、测试抓取的IP, 可用的保存在文件中

```
try:
    response = requests.get(PROXY_POOL_URL)
    if response.status_code == 200:
        return response.text
except ConnectionError:
    return None
```

私密代理

◦ 语法格式

```
1、语法结构
proxies = {
    '协议': '协议://用户名:密码@IP:端口号'
}

2、示例
proxies = {
    'http': 'http://用户名:密码@IP:端口号',
    'https': 'https://用户名:密码@IP:端口号'
}
```

示例代码


```
import requests
url = 'http://httpbin.org/get'
proxies = {
    'http': 'http://309435365:szayclhp@106.75.71.140:16816',
    'https': 'https://309435365:szayclhp@106.75.71.140:16816',
}
headers = {
    'User-Agent' : 'Mozilla/5.0',
}

html = requests.get(url,proxies=proxies,headers=headers,timeout=5).text
print(html)
```

##

- 非结构化数据保存

```
with open('xxx.jpg','wb') as f:
    f.write(res.content)
```

- 练习

- 1、将猫眼电影案例改写为 `requests` 模块实现
- 2、将电影天堂案例改写为 `requests` 模块实现
- 3、百度图片抓取：输入要抓取的图片内容,抓取首页的30张图片,保存到对应的文件夹,比如：
你想要谁的照片,请输入：赵丽颖
创建文件夹到指定目录：赵丽颖 并把首页30张图片保存到此文件夹下

百度图片练习代码实现

- 1、URL地址
`https://image.baidu.com/search/index?tn=baiduimage&word={}`
- 2、正则表达式
`"hoverURL": "(.*)"`

requests.post()参数

- 适用场景

Post类型请求的网站

- 参数-data

```
response = requests.post(url,data=data,headers=headers)
# data : post数据 (Form表单数据-字典格式)
```

- 请求方式的特点

一般

GET请求：参数在URL地址中有显示

POST请求：Form表单提交数据

(三) 动态加载数据抓取

• 打开方式及常用选项

1、打开浏览器，F12打开控制台，找到Network选项卡

2、控制台常用选项

1、Network：抓取网络数据包

1、ALL：抓取所有的网络数据包

2、XHR：抓取异步加载的网络数据包

3、JS：抓取所有的JS文件

2、Sources：格式化输出并打断点调试JavaScript代码，助于分析爬虫中一些参数

3、Console：交互模式，可对JavaScript中的代码进行测试

3、抓取具体网络数据包后

1、单击左侧网络数据包地址，进入数据包详情，查看右侧

2、右侧：

1、Headers：整个请求信息

General、Response Headers、Request Headers、Query String、Form Data

2、Preview：对响应内容进行预览

3、Response：响应内容

URL地址编码模块

模块名及导入

• 模块

模块名

urllib.parse

导入

```
import urllib.parse
```

```
from urllib import parse
```

• 作用

给URL地址中查询参数进行编码

编码前：https://www.baidu.com/s?wd=美女

编码后：https://www.baidu.com/s?wd=%E7%BE%8E%E5%A5%B3

常用方法

urllib.parse.urlencode({dict})

- URL地址中一个查询参数

```
# 查询参数: {'wd' : '美女'}
# urlencode编码后: 'wd=%e7%be%8e%e5%a5%b3'

# 示例代码
query_string = {'wd' : '美女'}
result = urllib.parse.urlencode(query_string)
# result: 'wd=%e7%be%8e%e5%a5%b3'
```

- URL地址中多个查询参数

```
from urllib import parse
params = {
    'wd' : '美女',
    'pn' : '50'
}
params = parse.urlencode(params)
url = 'http://www.baidu.com/s?{}'.format(params)
print(url)
```

- 拼接URL地址的3种方式

```
# 1、字符串相加
baseurl = 'http://www.baidu.com/s?'
params = 'wd=%E7XXXX&pn=20'
url = baseurl + params

# 2、字符串格式化（占位符）
params = 'wd=%E7XXXX&pn=20'
url = 'http://www.baidu.com/s?%s'% params

# 3、format()方法
url = 'http://www.baidu.com/s?{'
params = 'wd=#E7XXXX&pn=20'
url = url.format(params)
```

- 练习

在百度中输入要搜索的内容，把响应内容保存到本地文件

```
请输入搜索内容：赵丽颖
# 最终保存到本地文件 - 赵丽颖.html
```

代码实现

```
1、拼接URL地址
2、请求获取内容
3、保存到本地文件
with open(filename, 'w', encoding='utf-8') as f:
    f.write(html)
```

quote(string)编码

- 示例1

```
from urllib import parse

string = '美女'
print(parse.quote(string))
# 结果: %E7%BE%8E%E5%A5%B3
```

改写之前urlencode()代码，使用quote()方法实现

```
from urllib import parse

url = 'http://www.baidu.com/s?wd={}'
word = input('请输入要搜索的内容:')
query_string = parse.quote(word)
print(url.format(query_string))
```

unquote(string)解码

- 示例

```
from urllib import parse

string = '%E7%BE%8E%E5%A5%B3'
result = parse.unquote(string)
print(result)
```

总结

```
# 1、urllib.request
req = urllib.request.Request(url, headers)
res = urllib.request.urlopen(req)

# 2、响应对象res方法
html = res.read().decode()
url = res.geturl()
code = res.getcode()

# 3、urllib.parse
params = urllib.parse.urlencode({})
params = urllib.parse.quote('xxx')
```

```
string = urllib.parse.unquote('%E8XXXX')
```

多线程爬虫

应用场景

- 1、多进程：CPU密集程序
- 2、多线程：爬虫(网络I/O)、本地磁盘I/O

知识点回顾

- 队列

```
# 导入模块
from queue import Queue
# 使用
q = Queue()
q.put(url)
q.get() # 当队列为空时，阻塞
q.empty() # 判断队列是否为空，True/False
```

- 线程模块

```
# 导入模块
from threading import Thread

# 使用流程
t = Thread(target=函数名) # 创建线程对象
t.start() # 创建并启动线程
t.join() # 阻塞等待回收线程

# 如何创建多线程?????
t_list = []
for i in range(5):
    t = Thread(target=xxx)
    t_list.append(t)
    t.start()

for t in t_list:
    t.join()
```

线程锁

```
n = 2000

def f1():
    for i in range(5000):
        n += 1
```

```
def f1():
    for i in range(5000):
        n -= 1
```

正确执行

```
# n = n + 1
a = n + 1 # a=2001
n = a     # n=2001
# n = n - 1
a = n - 1 # a=2000
n = a     # n=2000
```

非正确执行

```
a = n + 1 #a=2001
a = n - 1 #a=1999
n = a     #n=1999
n = a     #n=1999
```

```
# 线程锁
from threading import Lock
lock = Lock()
lock.acquire()
## python代码 ##
lock.release()
```

解析模块

（一）正则解析模块

re模块使用流程

- 方法一

```
r_list=re.findall('正则表达式',html,re.S) #re.S 功能标志位 让点匹配到\n
```

- 方法二

```
# 1、创建正则编译对象
pattern = re.compile(r'正则表达式',re.S)
r_list = pattern.findall(html)
```

正则表达式元字符

元字符	含义
.	任意一个字符（不包括\n）
\d	一个数字
\s	空白字符

元字符	含义
\S	非空白字符
[]	包含[]内容
*	出现0次或多次
+	出现1次或多次

思考：请写出匹配任意一个字符的正则表达式？

```
import re
# 方法一
pattern = re.compile('.', re.S)
# 方法二
pattern = re.compile('[\s\S]')
```

贪婪匹配和非贪婪匹配

• 贪婪匹配（默认）

- 1、在整个表达式匹配成功的前提下,尽可能多的匹配 * + ?
- 2、表示方式: .* .+ .?

• 非贪婪匹配

- 1、在整个表达式匹配成功的前提下,尽可能少的匹配 * + ?
- 2、表示方式: .*? .+? .??

示例代码

```
import re

html = '''
<div><p>九霄龙吟惊天变</p></div>
<div><p>风云际会浅水游</p></div>
'''

# 贪婪匹配
pattern = re.compile('<div><p>.*</p></div>', re.S)
r = pattern.findall(html)
print(r)

# 非贪婪匹配 - 爬虫专用
pattern = re.compile('<div><p>.*?</p></div>', re.S)
r = pattern.findall(html)
print(r)
```

正则表达式分组

- 作用

在完整的模式中定义子模式，将每个圆括号中子模式匹配出来的结果提取出来

- 示例

```
import re

s = 'A B C D'
p1 = re.compile('\w+\s+\w+')
print(p1.findall(s))
# 结果: ['A B', 'C D']

p2 = re.compile('(\w+)\s+\w+')
print(p2.findall(s))
# 第1步(整体正则): ['A B', 'C D']
# 第2步(提取分组): ['A', 'C']

p3 = re.compile('(\w+)\s+(\w+)')
print(p3.findall(s))
# 第1步(整体正则): ['A B', 'C D']
# 第2步(提取分组): [('A', 'B'), ('C', 'D')]
```

- 分组总结

- 1、在网页中，想要什么内容，就加()
- 2、先按整体正则匹配，然后再提取分组()中的内容
如果有2个及以上分组()，则结果中以元组形式显示
[('小区1', '500万'), ('小区2', '600万'), ()]

- 练习

页面结构如下：

```
<div class="animal">.*?title="(.*?)".*?content">(.*?)</p>

<div class="animal">
  <p class="name">
    <a title="Tiger"></a>
  </p>
  <p class="content">
    Two tigers two tigers run fast
  </p>
</div>

<div class="animal">
  <p class="name">
    <a title="Rabbit"></a>
  </p>
```



```
<p class="content">
    Small white rabbit white and white
</p>
</div>
```

从以上html代码结构中完成如下内容信息的提取：

```
# 问题1
[('Tiger', ' Two...'), ('Rabbit', 'Small..')]
# 问题2
动物名称 : Tiger
动物描述 : Two tigers two tigers run fast
*****
动物名称 : Rabbit
动物描述 : Small white rabbit white and white
```

代码实现

```
'''
    正则表达式练习
'''
import re
html = '''<div class="animal">
    <p class="name">
        <a title="Tiger"></a>
    </p>
    <p class="content">
        Two tigers two tigers run fast
    </p>
</div>

<div class="animal">
    <p class="name">
        <a title="Rabbit"></a>
    </p>

    <p class="content">
        Small white rabbit white and white
    </p>
</div>'''

pattern = re.compile(r'.*?<a title="(.*?)">.*?<p class="content">(.*?)</p>', re.S)
print(pattern.findall(html))
```

(二) xpath解析

定义

XPath即为XML路径语言，它是一种用来确定XML文档中某部分位置的语言，同样适用于HTML文档的检索

示例

```
<ul class="CarList">
  <li class="bjd" id="car_001" href="http://www.bjd.com/">
    <p class="name">布加迪</p>
    <p class="model">威航</p>
    <p class="price">2500万</p>
    <p class="color">红色</p>
  </li>

  <li class="byd" id="car_002" href="http://www.byd.com/">
    <p class="name">比亚迪</p>
    <p class="model">秦</p>
    <p class="price">15万</p>
    <p class="color">白色</p>
  </li>
</ul>
```

匹配演示

- 1、查找所有的li节点
`//li`
- 2、获取所有汽车的名称：所有li节点下的子节点p的值（class属性值为name）
`//li/p[@class="name"]`
- 3、找比亚迪车的信息：获取ul节点下第2个li节点的汽车信息
`//ul/li[2]`
- 4、获取所有汽车的链接：ul节点下所有li子节点的href属性的值
`//ul/li/@href`

只要涉及到条件,加 []
只要获取属性值,加 @

选取节点

- 1、// ：从所有节点中查找（包括子节点和后代节点）
- 2、@ ：获取属性值
 - # 使用场景1（属性值作为条件）
`//div[@class="movie-item-info"]`
 - # 使用场景2（直接获取属性值）
获取所有电影详情页的链接
`//div[@class="movie-item-info"]/p[@class="name"]/a/@href`

匹配多路径 (或)

```
xpath表达式1 | xpath表达式2 | xpath表达式3
```

常用函数

```
1、contains() : 匹配属性值中包含某些字符串节点
# 查找id属性值中包含字符串 "car_" 的 li 节点
//li[contains(@id,"car_")]

2、text() : 获取节点的文本内容
# 查找所有汽车的价格
//li/p[@class="price"]/text()

# 得到字符串的两种方式:
1、text(): //li/p/text()
2、@href: //li/p/@href
```

lxml解析库

安装

```
sudo pip3 install lxml
```

使用流程

```
1、导模块
from lxml import etree

2、创建解析对象
parse_html = etree.HTML(html)

3、解析对象调用xpath
r_list = parse_html.xpath('xpath表达式')
```

html样本

```
<div class="wrapper">
  <a href="/" id="channel">新浪社会</a>
  <ul id="nav">
    <li><a href="http://domestic.sina.com/" title="国内">国内</a></li>
    <li><a href="http://world.sina.com/" title="国际">国际</a></li>
    <li><a href="http://mil.sina.com/" title="军事">军事</a></li>
    <li><a href="http://photo.sina.com/" title="图片">图片</a></li>
    <li><a href="http://society.sina.com/" title="社会">社会</a></li>
    <li><a href="http://ent.sina.com/" title="娱乐">娱乐</a></li>
    <li><a href="http://tech.sina.com/" title="科技">科技</a></li>
    <li><a href="http://sports.sina.com/" title="体育">体育</a></li>
    <li><a href="http://finance.sina.com/" title="财经">财经</a></li>
    <li><a href="http://auto.sina.com/" title="汽车">汽车</a></li>
  </ul>
</div>
```

示例+练习

```
from lxml import etree

html = '''
<div class="wrapper">
  <a href="/" id="channel">新浪社会</a>
  <ul id="nav">
    <li><a href="http://domestic.sina.com/" title="国内">国内</a></li>
    <li><a href="http://world.sina.com/" title="国际">国际</a></li>
    <li><a href="http://mil.sina.com/" title="军事">军事</a></li>
    <li><a href="http://photo.sina.com/" title="图片">图片</a></li>
    <li><a href="http://society.sina.com/" title="社会">社会</a></li>
    <li><a href="http://ent.sina.com/" title="娱乐">娱乐</a></li>
    <li><a href="http://tech.sina.com/" title="科技">科技</a></li>
    <li><a href="http://sports.sina.com/" title="体育">体育</a></li>
    <li><a href="http://finance.sina.com/" title="财经">财经</a></li>
    <li><a href="http://auto.sina.com/" title="汽车">汽车</a></li>
  </ul>
</div>'''

# 创建解析对象
parse_html = etree.HTML(html)

# 调用xpath返回结束,text()为文本内容
a_list = parse_html.xpath('//a/text()')
print(a_list)

# 提取所有的href的属性值

# 提取所有href的值,不包括 /

# 获取 图片、军事、...,不包括新浪社会
```

xpath最常使用方法

```
1、节点对象列表
# xpath示例: //div、//div[@class="student"]、//div/a[@title="stu"]/span

2、字符串列表
# xpath表达式中末尾为: @src、@href、text()
# //div/text() : div节点文本内容
# //div//text() : div节点+所有节点文本内容

1、基准xpath表达式: 得到节点对象列表
2、for r in [节点对象列表]:
    username = r.xpath('./xxxxxx')

# 此处注意遍历后继续xpath一定要以: . 开头, 代表当前节点
```

(三) json解析模块

json.loads(json)

- 作用

把json格式的字符串转为Python数据类型

- 示例

```
html_json = json.loads(res.text)
```

json.dumps(python)

- 作用

把 python 类型 转为 json 类型

- 示例

```
import json

# json.dumps()之前
item = {'name': 'QQ', 'app_id': 1}
print('before dumps', type(item)) # dict
# json.dumps之后
item = json.dumps(item)
print('after dumps', type(item)) # str
```

json.load(f)

作用

将json文件读取,并转为python类型

示例

```
import json

with open('D:\\spider_test\\xiaomi.json', 'r') as f:
    data = json.load(f)

print(data)
```

json.dump(python,f,ensure_ascii=False)

- 作用

把python数据类型 转为 json格式的字符串
一般让你把抓取的数据保存为json文件时使用

- 参数说明

第1个参数: python类型的数据(字典, 列表等)
第2个参数: 文件对象
第3个参数: ensure_ascii=False # 序列化时编码

- 示例1

```
import json

item = {'name':'QQ','app_id':1}
with open('小米.json','a') as f:
    json.dump(item,f,ensure_ascii=False)
```

- 示例2

```
import json

item_list = []
for i in range(3):
    item = {'name':'QQ','id':i}
    item_list.append(item)

with open('xiaomi.json','a') as f:
    json.dump(item_list,f,ensure_ascii=False)
```

json模块总结

爬虫最常用

- 1、数据抓取 - json.loads(html)
将响应内容由: json 转为 python
- 2、数据保存 - json.dump(item_list,f,ensure_ascii=False)
将抓取的数据保存到本地 json文件

抓取数据一般处理方式

- 1、txt文件
- 2、csv文件
- 3、json文件
- 4、MySQL数据库
- 5、MongoDB数据库
- 6、Redis数据库

数据持久化存储

数据持久化存储 - csv文件

- 作用

将爬取的数据存放到本地的csv文件中

- 使用流程

- 1、导入模块
- 2、打开csv文件
- 3、初始化写入对象
- 4、写入数据(参数为列表)

```
import csv

with open('film.csv','w') as f:
    writer = csv.writer(f)
    writer.writerow([])
```

- 示例代码

创建 test.csv 文件，在文件中写入数据

```
# 单行写入 (writerow([]))
import csv
with open('test.csv','w',newline='') as f: #newline='' 解决每一行出现空行的问题
    writer = csv.writer(f)
    writer.writerow(['步惊云','36'])
    writer.writerow(['超哥哥','25'])

# 多行写入(writerows([],[],[]))
import csv
with open('test.csv','w',newline='') as f:
    writer = csv.writer(f)
    writer.writerows([('聂风','36'),('秦霜','25'),('孔慈','30')])
```

- 练习

猫眼电影数据存入本地 maoyanfilm.csv 文件 - 使用writerow()方法实现

```
# 存入csv文件 - writerow()
def write_html(self, film_list):
    with open('film.csv', 'a') as f:
        # 初始化写入对象,注意参数f别忘了
        writer = csv.writer(f)
        for film in film_list:
            L = [
                film[0].strip(),
                film[1].strip(),
                film[2].strip()[5:15]
            ]
            # writerow()参数为列表
            writer.writerow(L)
```

思考：使用 writerows()方法实现？

```
# 存入csv文件 - writerows()
def write_html(self, film_list):
    L = []
    with open('film.csv', 'a') as f:
        # 初始化写入对象,注意参数f别忘了
        writer = csv.writer(f)
        for film in film_list:
            t = (
                film[0].strip(),
                film[1].strip(),
                film[2].strip()[5:15]
            )
            L.append(t)
        # writerows()参数为列表
        writer.writerows(L)
```

数据持久化存储 - MySQL数据库

1、在数据库中建库建表

```
# 连接到mysql数据库
mysql -h127.0.0.1 -uroot -p123456
# 建库建表
create database maoyandb charset utf8;
use maoyandb;
create table filmtab(
    name varchar(100),
    star varchar(300),
    time varchar(50)
)charset=utf8;
```

• 2、回顾pymysql基本使用

```
import pymysql
```



```

# 创建2个对象
db = pymysql.connect('localhost','root','123456','maoyandb',charset='utf8')
cursor = db.cursor()

# 执行SQL命令并提交到数据库执行
# execute()方法第二个参数为列表传参补位
ins = 'insert into filmtab values(%s,%s,%s)'
cursor.execute(ins,['霸王别姬','张国荣','1993'])
# 方法2
ins = "insert into filmtab values('%s','%s','%s')" % ('霸王别姬','张国荣','1993')
cursor.excute(ins)

db.commit()

# 关闭
cursor.close()
db.close()

```

- 来试试高效的executemany()方法?

```

import pymysql

# 创建2个对象
db = pymysql.connect('192.168.153.137','tiger','123456','maoyandb',charset='utf8')
cursor = db.cursor()

# 抓取的数据
film_list = [('月光宝盒','周星驰','1994'),('大圣娶亲','周星驰','1994')]

# 执行SQL命令并提交到数据库执行
# execute()方法第二个参数为列表传参补位
cursor.executemany('insert into filmtab values(%s,%s,%s)',film_list)
db.commit()

# 关闭
cursor.close()
db.close()

```

- 3、将电影信息存入MySQL数据库（尽量使用executemany方法）

```

# mysql - executemany([()],(),[])
def write_html(self, film_list):
    L = []
    ins = 'insert into filmtab values(%s,%s,%s)'
    for film in film_list:
        t = (
            film[0].strip(),
            film[1].strip(),
            film[2].strip()[5:15]
        )

```

```
L.append(t)

self.cursor.executemany(ins, L)
# 千万别忘了提交到数据库执行
self.db.commit()
```

```
import pymysql

# __init__(self):
self.db = pymysql.connect('IP',... ...)
self.cursor = self.db.cursor()
# write_data(self):
self.cursor.execute('sql',[data1])
self.cursor.executemany('sql',[(data1),(data2),(data3)])
self.db.commit()
# main(self):
self.cursor.close()
self.db.close()
```

数据持久化存储 - MongoDB数据库

pymongo操作mongodb数据库

```
import pymongo

# 1.数据库连接对象
conn=pymongo.MongoClient('localhost',27017)
# 2.库对象
db = conn['库名']
# 3.集合对象
myset = db['集合名']
# 4.插入数据
myset.insert_one({字典})
```

```
import pymongo

# __init__(self):
self.conn = pymongo.MongoClient('IP',27017)
self.db = self.conn['db_name']
self.myset = self.db['set_name']

# write_data(self):
self.myset.insert_one(dict)
self.myset.insert_many([{} ,{} ,{}])

# MongoDB - Command
>show dbs
>use db_name
>show collections
```

```
>db.collection_name.find().pretty()
>db.collection_name.count()
# 删除集合
>db.collection_name.drop()
# 删除库
>db.dropDatabase()
```

代码实现

1、一级页面正则

```
<div class="movie-item-info">.*?href="(.*?)".*?title="(.*?)".*?<p class="star">(.*?)</p>.*?class="releasetime">(.*?)</p>
```

2、二级页面正则 - 评论

```
<div class="comment-content">(.*?)</div>
```

3、二级页面正则 - 图面链接

```
<div class="img.*?"></div>
```

反爬

• 基于User-Agent反爬

1、发送请求携带请求头：headers={'User-Agent' : 'Mozilla/5.0 xxxxxx'}

2、多个请求随机切换User-Agent

1、定义列表存放大量User-Agent，使用random.choice()每次随机选择

2、定义py文件存放大量User-Agent，使用random.choice()每次随机选择

3、使用fake_useragent模块每次访问随机生成User-Agent

```
# sudo pip3 install fake_useragent
```

```
* from fake_useragent import UserAgent
```

```
* ua = UserAgent()
```

```
* user_agent = ua.random
```

```
* print(user_agent)
```

```
# 或者
```

```
agent = UserAgent().random
```

• 响应内容前端JS做处理反爬

1、html页面中可匹配出内容，程序中匹配结果为空

* 响应内容中嵌入js，对页面结构做了一定调整导致，通过查看网页源代码，格式化输出查看结构，更改xpath或者正则测试

2、如果数据出不来可考虑更换 IE 的User-Agent尝试，数据返回最标准

• 常见的反爬机制及处理方式

1、Headers反爬虫：Cookie、Referer、User-Agent

解决方案：通过F12获取headers,传给requests.get()方法

2、IP限制：网站根据IP地址访问频率进行反爬,短时间内限制IP访问

解决方案:

- 1、构造自己IP代理池,每次访问随机选择代理,经常更新代理池
- 2、购买开放代理或私密代理IP
- 3、降低爬取的速度

3、User-Agent限制：类似于IP限制

解决方案：构造自己的User-Agent池,每次访问随机选择

4、对响应内容做处理

解决方案：打印并查看响应内容,用xpath或正则做处理

cookie模拟登录

适用网站及场景

抓取需要登录才能访问的页面

cookie和session机制

http协议为无连接协议

cookie: 存放在客户端浏览器

session: 存放在web服务器

案例

(一) 百度贴吧数据抓取案例

要求

- 1、输入贴吧名称:赵丽颖吧
- 2、输入起始页:1
- 3、输入终止页:3
- 4、保存到本地文件

赵丽颖吧-第1页.html、赵丽颖吧-第2页.html ...

实现步骤

- 1、查看是否为静态页面

右键 - 查看网页源代码 - 搜索数据关键字

- 2、找URL规律

第1页:http://tieba.baidu.com/f?kw=? ? &pn=0

第2页:http://tieba.baidu.com/f?kw=? ? &pn=50

第n页:pn=(n-1)*50

- 3、获取网页内容
- 4、提取所需数据
- 5、保存(本地文件、数据库)

代码实现

```
from urllib import request, parse
import time
import random
from useragents import ua_list

class TiebaSpider(object):
    def __init__(self):
        self.url = 'http://tieba.baidu.com/f?{'

    # 请求
    def get_html(self, url):
        req = request.Request(
            url=url,
            headers={'User-Agent': random.choice(ua_list)}
        )
        res = request.urlopen(req)
        html = res.read().decode('utf-8', 'ignore')

        return html

    # 解析
    def parse_html(self):
        pass

    # 保存
    def save_html(self, filename, html):
        with open(filename, 'w') as f:
            f.write(html)

    # 入口函数
    def run(self):
        name = input('请输入贴吧名:')
        begin = int(input('请输入起始页:'))
        stop = int(input('请输入终止页:'))
        # 拼接地址,发请求
        for page in range(begin, stop+1):
            pn = (page-1)*50
            params = {
                'kw' : name,
                'pn' : str(pn)
            }
            params = parse.urlencode(params)
            url = self.url.format(params)
            html = self.get_html(url)
            filename = '{}-第{}页.html'.format(name, page)
```

```

        self.save_html(filename,html)
        # 提示
        print('第%d页抓取成功' % page)
        # 每爬取1个页面随机休眠1-3秒钟
        time.sleep(random.randint(1,3))
        time.sleep(random.uniform(0,1))

if __name__ == '__main__':
    start = time.time()
    spider = TiebaSpider()
    spider.run()
    end = time.time()
    print('执行时间:%.2f' % (end-start))

```

ConnectionReset error [rror 104] :远程主机。。。 大概意思是你的访问被服务区判定为恶意攻击

方法：

while True:

try:

write_html(url,word)

break

except Exception as e:

time.sleep(0.5)

(二) 电影天堂二级页面抓取案例

领取任务

```

# 地址
电影天堂 - 2019年新片精品 - 更多

# 目标
电影名称、下载链接

# 分析
*****一级页面需抓取*****
    1、电影详情页链接

*****二级页面需抓取*****
    1、电影名称
    2、电影下载链接

```

实现步骤

- 1、确定响应内容中是否存在所需抓取数据
- 2、找URL规律

第1页 : https://www.dytt8.net/html/gndy/dyzz/list_23_1.html

第2页 : https://www.dytt8.net/html/gndy/dyzz/list_23_2.html

第n页 : https://www.dytt8.net/html/gndy/dyzz/list_23_n.html

- 3、写正则表达式

1、一级页面正则表达式

```
<table width="100%" .*?<td width="5%" .*?<a href="(.*?)".*?u!ink">.*?</table>
```

2、二级页面正则表达式

```
<div class="title_all"><h1><font color=#07519a>(.*?)</font></h1></div>.*?<td style="WORD-WRAP.*?>.*?>(.*?)</a>
```

- 4、把电影天堂数据存入MySQL数据库 - 增量爬取

思路

1、MySQL中新建表 request_finger,存储所有爬取过的链接的指纹

2、在爬取之前,先判断该指纹是否爬取过,如果爬取过,则不再继续爬取

5、代码实现

建库建表

```
create database filmskydb charset utf8;
use filmskydb;
create table urltab(
finger char(32)
)charset=utf8;
create table filmtab(
name varchar(200),
download varchar(500)
)charset=utf8;
```

周期性计划任务

crontab -e

* * * * * Linux命令

分时日月周

(三) 链家二手房案例 (xpath)

实现步骤

- 确定是否为静态

打开二手房页面 -> 查看网页源码 -> 搜索关键字

- xpath表达式

1、基准xpath表达式(匹配每个房源信息节点列表)

此处滚动鼠标滑轮时,li节点的class属性值会发生变化,通过查看网页源码确定xpath表达式

```
//ul[@class="sellListContent"]/li[@class="clear LOGVIEWDATA LOGCLICKDATA"]
```

2、依次遍历后每个房源信息xpath表达式

* 名称: `'./a[@data-el="region"]/text()'`

户型+面积+方位+是否精装

```
info_list = './div[@class="houseInfo"]/text()' [0].strip().split('|')
```

* 户型: `info_list[1]`

* 面积: `info_list[2]`

* 方位: `info_list[3]`

* 精装: `info_list[4]`

* 楼层: `'./div[@class="positionInfo"]/text()'`

* 区域: `'./div[@class="positionInfo"]/a/text()'`

* 总价: `'./div[@class="totalPrice"]/span/text()'`

* 单价: `'./div[@class="unitPrice"]/span/text()'`

(四) 百度贴吧图片抓取

目标思路

- 目标

抓取指定贴吧所有图片

- 思路

1、获取贴吧主页URL,下一页,找到不同页的URL规律

2、获取1页中所有帖子URL地址: [帖子链接1,帖子链接2,...]

3、对每个帖子链接发请求,获取图片URL

4、向图片的URL发请求,以wb方式写入本地文件

实现步骤

- 贴吧URL规律

`http://tieba.baidu.com/f?kw=??&pn=50`

- xpath表达式

1、帖子链接xpath

```
//div[@class="t_con cleafix"]/div/div/div/a/@href
```

2、图片链接xpath

```
//div[@class="d_post_content j_d_post_content  
clearfix"]/img[@class="BDE_Image"]/@src
```

3、视频链接xpath

```
//div[@class="video_src_wrapper"]/embed/@data-video
```

注意：此处视频链接前端对响应内容做了处理,需要查看网页源代码来查看，复制HTML代码在线格式化

代码实现

```
class BaiduImageSpider(object):
```

```
    def __init__(self):
```

```
        pass
```

```
    xxx
```

```
    def parse_html(self,url):
```

```
        xxxx
```

```
        t_list = ['/p/2323','/p/232323']
```

```
        for t in t_list:
```

```
            把1个t中所有的图片链接提取，并保存本地
```

```
            self.save_image(t)
```

```
http://imgsrc.baidu.com/forum/w%3D580/sign=d53167bca91ea8d38a22740ca70b30cf/73708c82  
d158ccbf8a4a653d16d8bc3eb0354176.jpg
```

(五) 有道翻译破解案例(post)

- 目标

破解有道翻译接口，抓取翻译结果

结果展示

请输入要翻译的词语：elephant

翻译结果：大象

请输入要翻译的词语：喵喵叫

翻译结果：mews

- 实现步骤

1. 打开首页
2. 准备抓包：F12开启控制台
3. 寻找地址

页面中输入翻译单词，控制台中抓取到网络数据包，查找并分析返回翻译数据的地址

4. 发现规律

找到返回具体数据的地址，在页面中多输入几个单词，找到对应URL地址，分析对比 **Network - All**(或者XHR) - **Form Data**，发现对应的规律

5. 寻找JS文件

右上角 ... -> **Search** -> 搜索关键字 -> 单击 -> 跳转到**Sources**，左下角格式化符号{}

6. 查看JS代码

搜索关键字，找到相关加密方法

7. 断点调试

8. 完善程序

- **具体实现**

- **1、开启F12抓包，找到Form表单数据如下：**

```
#form_data
i: 老鼠
from: auto
to:
dictResult: true
keyid: webfanyi
sign: 20806e51cef9d145ca5e54d229e1e898
client: fanyideskweb
product: webfanyi
appVersion: 1.0.0
vendor: web
pointParam: client,mysticTime,product
mysticTime: 1700394419689
keyfrom: fanyi.web
mid: 1
screen: 1
model: 1
network: wifi
abtest: 0
yduuid: abcdefg
```

```
#headers
```

```
Accept:
application/json, text/plain, */*
Accept-Encoding:
gzip, deflate, br
Accept-Language:
zh-CN,zh;q=0.9
Connection:
keep-alive
Content-Length:
312
Content-Type:
```

```
application/x-www-form-urlencoded
Cookie:
OUTFOX_SEARCH_USER_ID=-1694106329@10.108.162.134;
OUTFOX_SEARCH_USER_ID_NCOO=581254075.7213335
Host:
dict.youdao.com
Origin:
https://fanyi.youdao.com
Referer:
https://fanyi.youdao.com/
Sec-Ch-Ua:
"Google Chrome";v="119", "Chromium";v="119", "Not?A_Brand";v="24"
Sec-Ch-Ua-Mobile:
?0
Sec-Ch-Ua-Platform:
"Windows"
Sec-Fetch-Dest:
empty
Sec-Fetch-Mode:
cors
Sec-Fetch-Site:
same-site
User-Agent:
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/119.0.0.0 Safari/537.36
```

- 2、在页面中多翻译几个单词，观察Form表单数据变化

```
salt: 15614112641250
sign: 94008208919faa19bd531acde36aac5d
ts: 1561411264125
bv: f4d62a2579ebb44874d7ef93ba47e822
# 但是bv的值不变
```

- 3、一般为本地js文件加密，刷新页面，找到js文件并分析JS代码

```
# 方法1
Network - JS选项 - 搜索关键词salt
# 方法2
控制台右上角 - Search - 搜索salt - 查看文件 - 格式化输出
# 最终找到相关JS文件 : fanyi.min.js
```

- 4、打开JS文件，分析加密算法，用Python实现

```
# ts : 经过分析为13位的时间戳，字符串类型
js代码实现: "" + (new Date).getTime()
python实现: str(int(time.time()*1000))

# salt
js代码实现: ts+parseInt(10 * Math.random(), 10);
```

```
python实现: ts+str(random.randint(0,9))
```

```
# sign (设置断点调试, 来查看 e 的值, 发现 e 为要翻译的单词)
```

```
js代码实现: n.md5("fanyideskweb" + e + salt + "n%A-rKaT5fb[Gy?;N5@Tj")
```

```
python实现:
```

```
from hashlib import md5
string = ''
s = md5()
s.update(string.encode())
sign = s.hexdigest()
```

增量爬取思路

- 1、将爬取过的地址存放到数据库中
- 2、程序爬取时先到数据库中查询比对, 如果已经爬过则不会继续爬取

动态加载网站数据抓取

- 1、F12打开控制台, 页面动作抓取网络数据包
 - 2、抓取json文件URL地址
- ```
控制台中 XHR : 异步加载的数据包
XHR -> Query String Parameters(查询参数)
```

## (六) 民政部网站数据抓取

### 目标

- 1、URL: <http://www.mca.gov.cn/> - 民政数据 - 行政区划代码  
即: <http://www.mca.gov.cn/article/sj/xzqh/2019/>
- 2、目标: 抓取最新中华人民共和国县以上行政区划代码

### 实现步骤

- 1、从民政数据网站中提取最新行政区划代码链接

```
特点
1、最新的在上面
2、命名格式: 2019年x月中华人民共和国县以上行政区划代码
a_list: [<element a at xxx>,<element a at xxx>]
a_list = p.xpath('//a[@class="articlelist"]')
for a in a_list:
 title = a.get('title')
 if title.endswith('行政区划代码'):
 # 第一个满足条件的一定是最新的文章链接
 href = a.get('href')
 break
```

- 2、从二级页面链接中提取真实链接 (反爬-响应内容中嵌入JS, 指向新的链接)

- 1、向二级页面链接发请求得到响应内容，并查看嵌入的JS代码
- 2、正则提取真实的二级页面链接

### 3、真实链接中提取所需数据

### 4、代码实现

```
xpath表达式
//a[@class="artitlelist"]/@href

提取真实链接的正则表达式
window.location.href="(.*?)"
```

### 扩展

- 1、建立增量爬虫 - 网站有更新时抓取，否则不抓
- 2、所抓数据存到数据库，按照层级关系分表存储 - 省、市、县表

## (七) 豆瓣电影数据抓取案例

### • 目标

- 1、地址：豆瓣电影 - 排行榜 - 剧情
- 2、目标：电影名称、电影评分

### • F12抓包 (XHR)

- 1、Request URL(基准URL地址) : [https://movie.douban.com/j/chart/top\\_list?](https://movie.douban.com/j/chart/top_list?)
- 2、Query String(查询参数)

```
抓取的查询参数如下：
type: 13 # 电影类型
interval_id: 100:90
action: ''
start: 0 # 每次加载电影的起始索引值 0 20 40 60
limit: 20 # 每次加载的电影数量
```

## (八) 腾讯招聘数据抓取

### • 确定URL地址及目标

- 1、URL：百度搜索腾讯招聘 - 查看工作岗位
- 2、目标：职位名称、工作职责、岗位要求

### • 要求与分析

- 1、通过查看网页源码,得知所需数据均为 Ajax 动态加载
- 2、通过F12抓取网络数据包,进行分析
- 3、一级页面抓取数据: PostID
- 4、二级页面抓取数据: 职位名称、工作职责、岗位要求

- 一级页面json地址(index在变,timestamp未检查)

```
https://careers.tencent.com/tencentcareer/api/post/Query?
timestamp=1563912271089&countryId=&cityId=&bgIds=&productId=&categoryId=&parentCategoryId=&attrId=&keyword=&pageIndex={}&pageSize=10&language=zh-cn&area=cn
```

- 二级页面地址(postId在变,在一级页面中可拿到)

```
https://careers.tencent.com/tencentcareer/api/post/ByPostId?
timestamp=1563912374645&postId={}&language=zh-cn
```

## (九) 小米应用商店抓取(多线程)

### 目标

- 1、网址 : 百度搜 - 小米应用商店, 进入官网
- 2、目标 : 所有应用分类  
应用名称  
应用链接

### 实现步骤

- 1、确认是否为动态加载

- 1、页面局部刷新
  - 2、右键查看网页源代码, 搜索关键字未搜到
- # 此网站为动态加载网站, 需要抓取网络数据包分析

- 2、F12抓取网络数据包

- 1、抓取返回json数据的URL地址 (Headers中的Request URL)  
`http://app.mi.com/categoryAllListApi?page={}&categoryId=2&pageSize=30`
- 2、查看并分析查询参数 (headers中的Query String Parameters)  
page: 1  
categoryId: 2  
pageSize: 30  
# 只有page在变, 0 1 2 3 ... , 这样我们就可以通过控制page的值拼接多个返回json数据的URL地址

- 将抓取数据保存到csv文件

```
注意多线程写入的线程锁问题
from threading import Lock
lock = Lock()
加锁
lock.acquire()
python语句
释放锁
lock.release()
```

- 整体思路

```
假如说存json
1、在 __init__(self) 中创建文件对象，多线程操作此对象进行文件写入
self.f = open('xiaomi.json','w')
self.item_list = []
2、把所有数据抓取完成后,存入到json文件
for app in xxx:
 item['name'] = app['xxx']
 self.item_list.append(item)
3、所有数据抓取完成关闭文件
程序最后：
json.dump(self.item_list,self.f,ensure_ascii=False)
```

- 代码实现

```
正则 [(2,'聊天社交'),(3,'图书')]
(.*?)
```

## (十) 人人网登录案例

- 方法一 - 登录网站手动抓取Cookie

```
1、先登录成功1次,获取到携带登录信息的Cookie
 登录成功 - 个人主页 - F12抓包 - 刷新个人主页 - 找到主页的包(profile)
2、携带着cookie发请求
 ** Cookie
 ** User-Agent
```

```
1、将self.url改为 个人主页的URL地址
2、将Cookie的值改为 登录成功的Cookie值
import requests
from lxml import etree

class RenrenLogin(object):
 def __init__(self):
 self.url = 'xxxxxxx'
 self.headers = {
 'Cookie': 'xxxxxx',
```

```

 'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/76.0.3809.100 Safari/537.36'
 }

 def get_html(self):
 html = requests.get(url=self.url, headers=self.headers).text
 self.parse_html(html)

 def parse_html(self, html):
 parse_html = etree.HTML(html)
 r_list = parse_html.xpath('//*[@id="operate_area"]/div[1]/ul/li[1]/span/text()')
 print(r_list)

if __name__ == '__main__':
 spider = RenrenLogin()
 spider.get_html()

```

## • 方法二

原理

- 1、把抓取到的cookie处理为字典
- 2、使用requests.get()中的参数:cookies

处理cookie为字典

```
处理cookies为字典
```

代码实现

```
11
```

## • 方法三 - requests模块处理Cookie

原理思路及实现

# 1. 思路

requests模块提供了session类,来实现客户端和服务端的会话保持

# 2. 原理

1、实例化session对象

```
session = requests.session()
```

2、让session对象发送get或者post请求

```
res = session.post(url=url, data=data, headers=headers)
```

```
res = session.get(url=url, headers=headers)
```

# 3. 思路梳理

浏览器原理: 访问需要登录的页面会带着之前登录过的cookie

程序原理: 同样带着之前登录的cookie去访问 - 由session对象完成



- 1、实例化session对象
- 2、登录网站：session对象发送请求,登录对应网站,把cookie保存在session对象中
- 3、访问页面：session对象请求需要登录才能访问的页面,session能够自动携带之前的这个cookie,进行请求

## 具体步骤

- 1、寻找Form表单提交地址 - 寻找登录时POST的地址

查看网页源码,查看form表单,找action对应的地址: `http://www.renren.com/PLogin.do`

- 2、发送用户名和密码信息到POST的地址

\* 用户名和密码信息以什么方式发送? -- 字典

键 : `<input>`标签中name的值(email,password)

值 : 真实的用户名和密码

`post_data = {'email': '', 'password': ''}`

```
session = requests.session()
```

```
session.post(url=url,data=data)
```